

BME280 temperature sensor on the Raspberry Pi 5

Gauge display in Mathematica, readout in Python

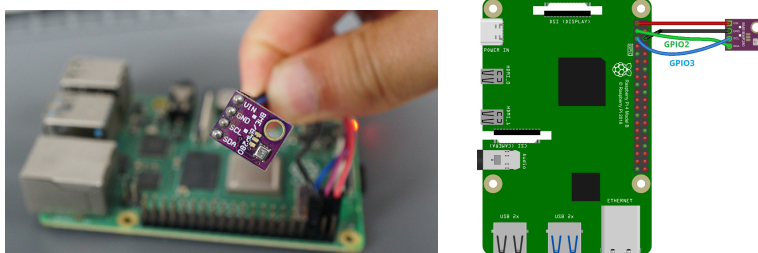
Source: <https://www.21stcenturycoding.net/sensors/#TemperatureGauge>

In this notebook we will interactively display the readings of the common temperature sensor BME280 on a temperature gauge.

There are many tutorials, but most only give a list of numbers on the terminal, and are not graphical. Here we learn how to exploit Python's low level access and Mathematica's high level interaction, processing and display.

This cheap and widely used sensor communicates using the I2C communication protocol, so the wiring is very simple .

To connect your sensor to the Pi, and how to enable the I2C protocol on your RPI5, see the clear explanations on: <https://randomnerdtutorials.com/raspberry-pi-bme280-python/>, from which also the images below were taken.



You can use the default Raspberry Pi I2C pins as shown in the following table:

BME280	→	Raspberry Pi
Vin	→	3.3V
GND	→	GND
SCL	→	GPIO 3
SDA	→	GPIO 2

Enable the I2C as follows:

- Click on the Raspberry Pi logo on the upper left

- Preferences → Raspberry Pi Configuration → Interfaces
- Turn on I2C, press OK.

We will use Python to read the sensor. We therefore start a Python session from within Mathematica.

This creates a so-called virtual environment, in which we can safely work. All commands into this virtual environment can be given from Mathematica, and all results can be easily imported into Mathematica. That is exactly what we go do.

When this is done for the first time, you will see that Mathematica automatically downloads all required packages.

To read the BME280 sensor data, we need a special package, RPI.BME280.

Install this RPI.BME280 package in the virtual Python environment:

```
In[1]:= session = StartExternalSession[
  <|"System" → "Python",
  "Evaluator" → <|"Dependencies" → {"RPI.BME280"}|>,
  "SessionProlog" → "import time"|>]
```

```
Out[1]= ExternalSessionObject[ System: Python SessionID: 590a2efc-2230-472
EvaluatorID: b16f6525-9b54-bd46-b061-99c26424b543 Dependencies: {numpy, zmq, I
```

Curious? Click on the + and see the details of this ExternalSessionObject.

As you see, all is done automatically.

Now we can create and 'ExternalEvaluate' the Python code to initialize the bus, and load the calibration parameters:

```
ExternalEvaluate[session, "
import smbus2
import bme280

# BME280 sensor address (default address)
address = 0x76

# Initialize I2C bus
bus = smbus2.SMBus(1)

# Load calibration parameters
calibration_params = bme280.load_calibration_params(bus, address)
"]
```

We now define a *function* in Python that can be called by Mathematica (as usual in Mathematica: with double square brackets around the arguments, or only double square brackets if there are no arguments). The function is named 'readSensor'. Note that we use the standard *delayed assignment* `:=` to define the function. This guarantees that the function definition is created only when we call it (using only `=` would give the wrong function if it was e.g. supposed to return the current time; it would always return the time of its first creation ...):

```
In[4]:= readSensor := ExternalEvaluate[session, "
# Read sensor data
data = bme280.sample(bus, address, calibration_params)
def read():
    temperature_celsius = data.temperature
    pressure = data.pressure
    humidity = data.humidity
    return (temperature_celsius, pressure, humidity)
"]
```

The function can now be called, and assigned to the three variables that it returns. The triple of measurements is stored in a small list, enclosed as usual by curly brackets:

```
In[6]:= {temperature, pressure, humidity} = readSensor[]
Out[6]= {18.5268, 1032.29, 58.7412}
```

Nice! The data have been measured, and are now available in Mathematica.

Some extra info

A fancy weather station - the first steps

Display on a gauge is easy in Mathematica. There are some nice pre-programmed gauges. You can see this by asking the Help function (the question mark **?**) with wildcards *****:

```
In[7]:= ? * Gauge *
```

```
Out[7]=
```

System

AngularGauge	GaugeFrameElementFunction	GaugeStyle
BulletGauge	GaugeFrameSize	HorizontalGauge
ClockGauge	GaugeFrameStyle	ThermometerGauge
GaugeFaceElementFunction	GaugeLabels	VerticalGauge
GaugeFaceStyle	GaugeMarkers	

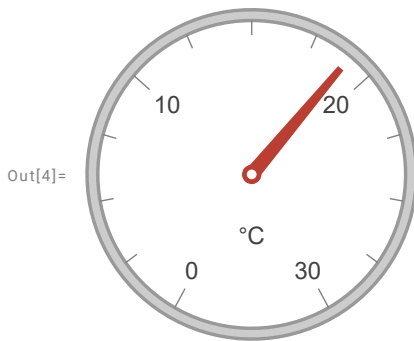
To get more information on e.g. AngularGauge (which we will use below), click on it, and ask for more information by clicking the small encircled **i** in the upper right corner to open the Wolfram Documentation Center. You find lots of examples of how to use this function.

A strong advice: use this Documentation Center as often as you can. Wolfram's documentation is without hesitation world-class, maybe even the best that exists for computer code. All examples are lines of code that you can directly use in your application. And maybe most important (at least it was for me using this system for the last 30 years): IT IS FUN, you regularly are pleasantly surprised.

We go use the example with units, and take this cell from the Documentation:

Of course, it is much more 21st century coding if it is displayed in the way we are used to see: Quantity[19, "DegreesCelsius"] should be displayed as **19 °C**. For this we can use Mathematica's 'Evaluate in place': only interpret the highlighted text, and press `SHIFT` `CTRL` `ENTER`. Try it:

```
In[4]:= AngularGauge[Quantity[19, "DegreesCelsius"],  
  {Quantity[0, "DegreesCelsius"], Quantity[30, "DegreesCelsius"]}, GaugeLabels → "Unit"]
```



The entry should now have changed to:

```
AngularGauge[19 °C, {0 °C, 30 °C}, GaugeLabels → "Unit"]
```

Much nicer. I really can recommend to adopt this 21st century coding style in all your next coding adventures.

Now code becomes much easier explainable to others, faster to grab when you see the code again after some time, and is no longer so old-fashioned.

Updating the gauge continuously: real-time measurement

The Dynamic function in Mathematic is the equivalent of 'keep doing it continuously'.

The code below gives a first small display of your weather station. Touch the sensor with your fingers, and see what happens. Study the code to easily see how it was stylized with a nice font, different gauges and colors.

```

In[3]:= Dynamic[
  temp = Quantity[First@readSensor[], "DegreesCelsius"];
  GraphicsRow[
    {" ",
     AngularGauge[temp, {15, 30}, GaugeLabels → "Unit"],
     ThermometerGauge[temp, {15, 30}, GaugeLabels → "Unit"],
     Framed[Text[Style[NumberForm[First@readSensor[], {Infinity, 2}],
       96, FontFamily → "Kalam"]], ImageSize → {270, 120}]
    ],
  ImageSize → 600, Background → LightBlue,
  Frame → True, FrameStyle → Directive[Thickness[0.015], Blue]
]

```

Out[14]=



Adapt the measurement range to your requirements.

Want some other fonts?

The following command uses `$FontFamilies` to give the list of all fonts on your Raspberry Pi 5, and The **Style[#]&** function prints each of the font names. The symbol **#** is the 'Slot' of the function, where the entry goes in, and **&** denotes that it is a 'pure function'. The notation **/'@** is short form for the function `Map`: do the function for all members of the list. With the function **Style[#, FontFamily→#]&** the printout is in the appearance of that font. It could not be done shorter.

```
In[81]:= Style[#, FontFamily → #] & /@$FontFamilies
```

```
Out[81]=
```

```
{ ALEGREYA SC, Bitstream Charter, Bitstream Vera Sans, Bitstream Vera Sans Mono,
  Bitstream Vera Serif, C059 [UKWN], C059 [urw], Cantarell, Cantarell Extra Bold,
  Cantarell Light, Cantarell Thin, Clear Sans, Clear Sans Light, Clear Sans Medium,
  Clear Sans Thin, Courier, Cousine, D050000L [urw], D050000L [URW ],
  DejaVu Math TeX Gyre, DejaVu Sans, DejaVu Sans Condensed, DejaVu Sans Mono,
  DejaVu Serif, DejaVu Serif Condensed, Droid Sans Fallback, Droid Serif, EB Garamond,
  EB Garamond o8, EB GARAMOND 12 ALL SC, EB Garamond SC, EB GARAMOND SC o8, Economica, Felipa,
  FreeMono, FreeSans, FreeSerif, Gentium Basic, Inconsolata, Kalam, Kalam Bold, Kalam Light,
  Lato, Lato Black, Lato Hairline, Lato Light, League Gothic, League Gothic Condensed, League Gothic Condensed Italic,
  Liberation Mono, Liberation Sans, Liberation Sans Narrow, Liberation Serif,
  Monospace, Nimbus Mono PS [UKWN], Nimbus Mono PS [urw], Nimbus Roman [UKWN],
  Nimbus Roman [urw], Nimbus Sans Narrow [UKWN], Nimbus Sans Narrow [urw],
  Nimbus Sans [UKWN], Nimbus Sans [urw], Nimbus Sans [URW ], Noto Mono,
  Noto Sans Mono, Oswald, P052 [UKWN], P052 [urw], Piboto Condensed, Piboto [Goog],
  Piboto Light, PibotoLt, Piboto [pyrs], Piboto Thin, Playfair Display, Playfair Display Black,
  Quicksand, Quicksand Light, Quicksand Medium, Roboto, Roboto Black, Roboto Condensed,
  Roboto Condensed Light, Roboto Light, Roboto Medium, Roboto Slab, Roboto Thin,
  Sans Serif, Serif, Shadows Into Light Two, Source Code Pro, Source Code Pro Black,
  Source Code Pro ExtraLight, Source Code Pro Light, Source Code Pro Medium,
  Source Code Pro Semibold, Source Sans Code, Source Sans Code SemBd, Source Sans Pro,
  Source Sans Pro Black, Source Sans Pro ExtraLight, Source Sans Pro Light, Source Sans Pro SemiBold,
  Source Serif Pro, Source Serif Pro Semibold, Standard Symbols PS [urw],
  Standard Symbols PS [URW ], Swiss 721, Titillium Web, Twitter Color Emoji,
  URW Bookman [UKWN], URW Bookman [urw], URW Gothic [UKWN], URW Gothic [urw],
  Utopia, Yanone Kaffeesatz, Yanone Kaffeesatz Extra Light, Yanone Kaffeesatz Light, Z003 [UKWN], Z003 [urw] }
```

From here it is just your imagination to make a really fancy weather station.

Of course, a weather station with gauges can also be programmed in other languages, see e.g. here. But this code is far more complex, and most important, takes far more time to implement than in Mathematica.